

UNIT 3

PROPERTIES OF REGULAR LANGUAGES

OVERVIEW

- Regular Expression (RE)
- Operators of RE
- Building RE
- Languages of RE
- $FA \longrightarrow RE$
- $RE \longrightarrow FA$
- **PROPERTIES OF RL**
- **PUMPING LEMMA**

Properties of RL

- **Closure properties-** Useful tool for building complex Automata
- **Decision Properties-** Deciding whether two automata define the same language.
- **Proving languages not be regular-** Pumping lemma is a powerful tool for proving certain languages are not regular.

Decision Properties of RL

- A problem is said to be decidable if $\exists$ an algorithm to solve the problem.
- A problem is said to be undecidable if there is no algorithm to solve the problem.

The following problems are decidable under R.L.

1. Emptiness problem.
2. Finiteness problem.
3. Equivalence problem.

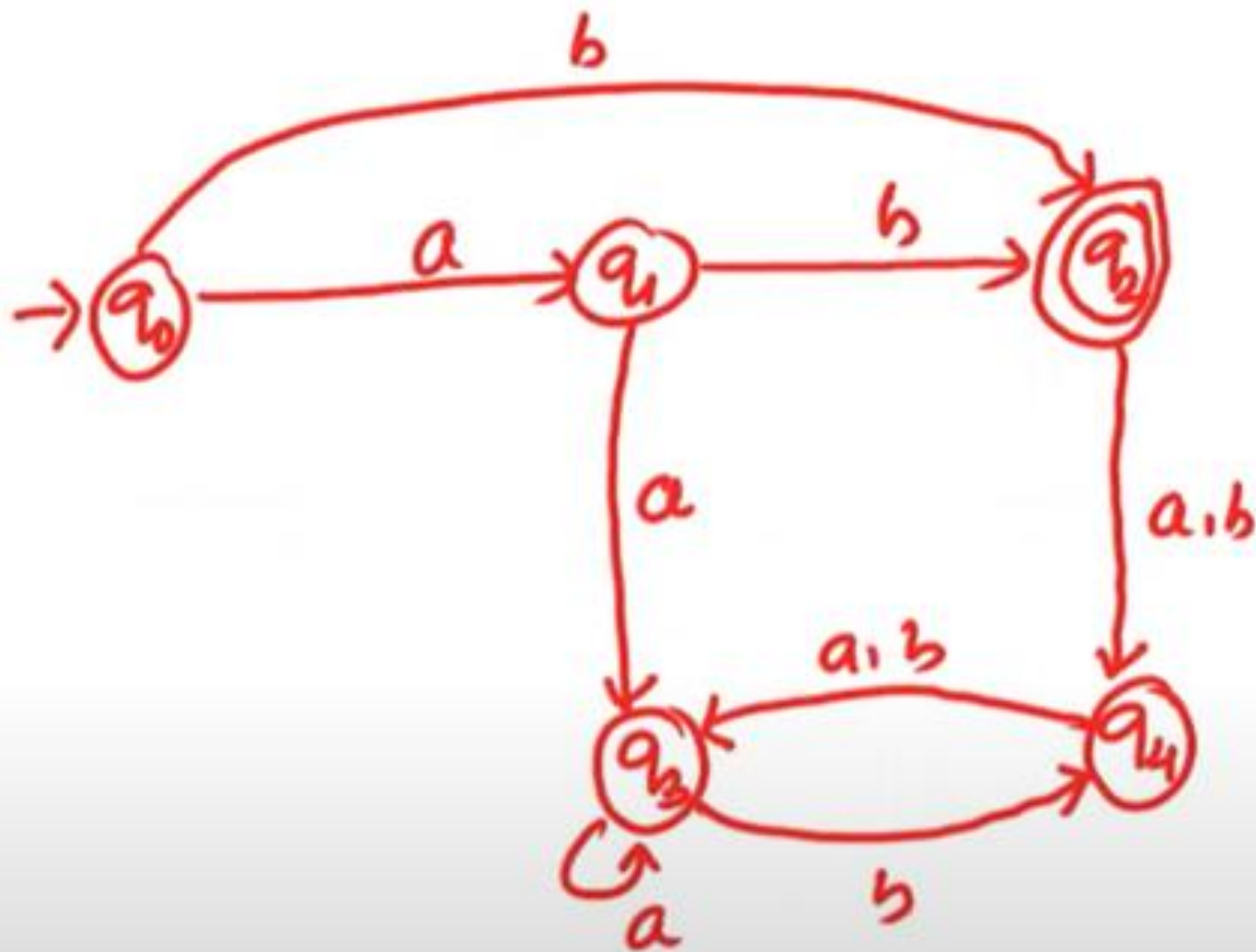
① Emptyness problem

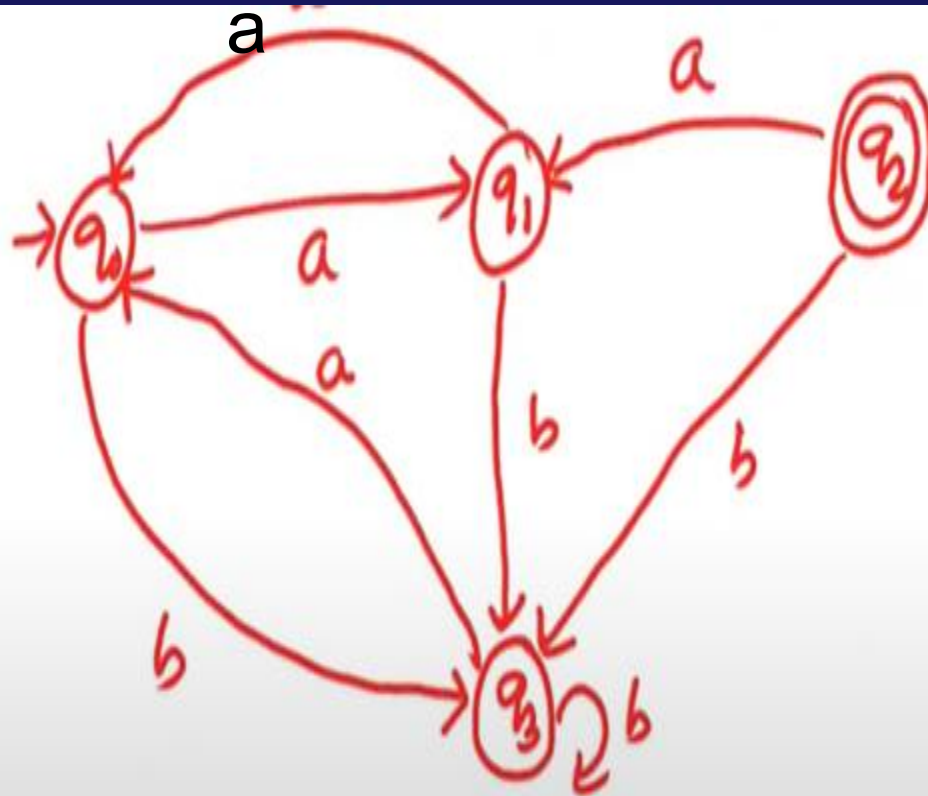
Checking whether a given F.A. accepts empty language or not.

Algorithm

S1. Eliminate all inaccessible states from the given F.A.

S2. In the remaining F.A. if there is at least one final state then the given F.A. accepts non empty language otherwise empty language.





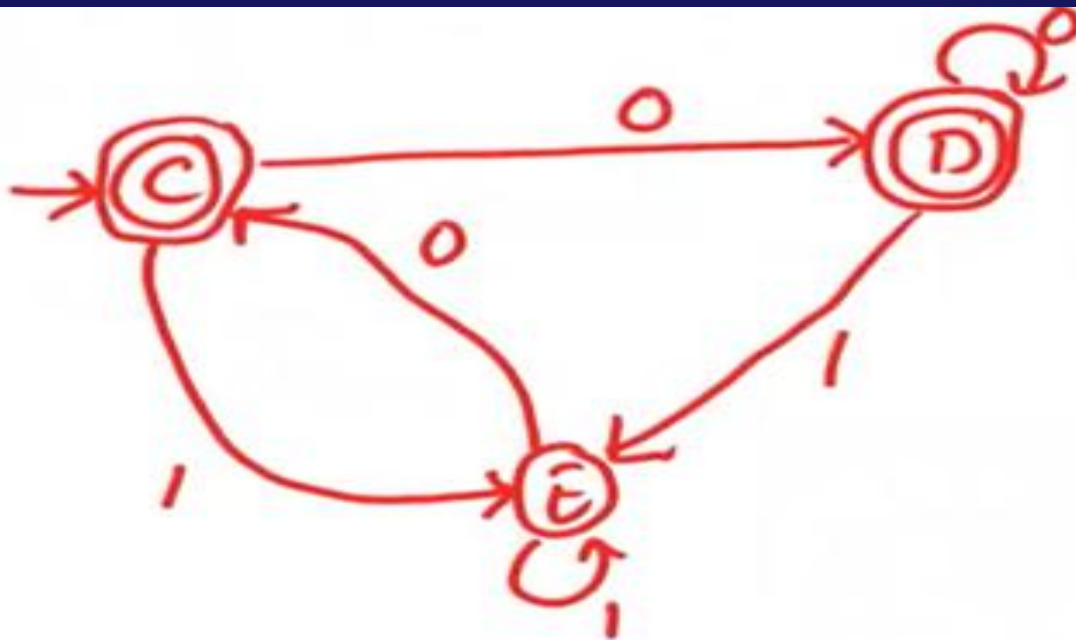
4

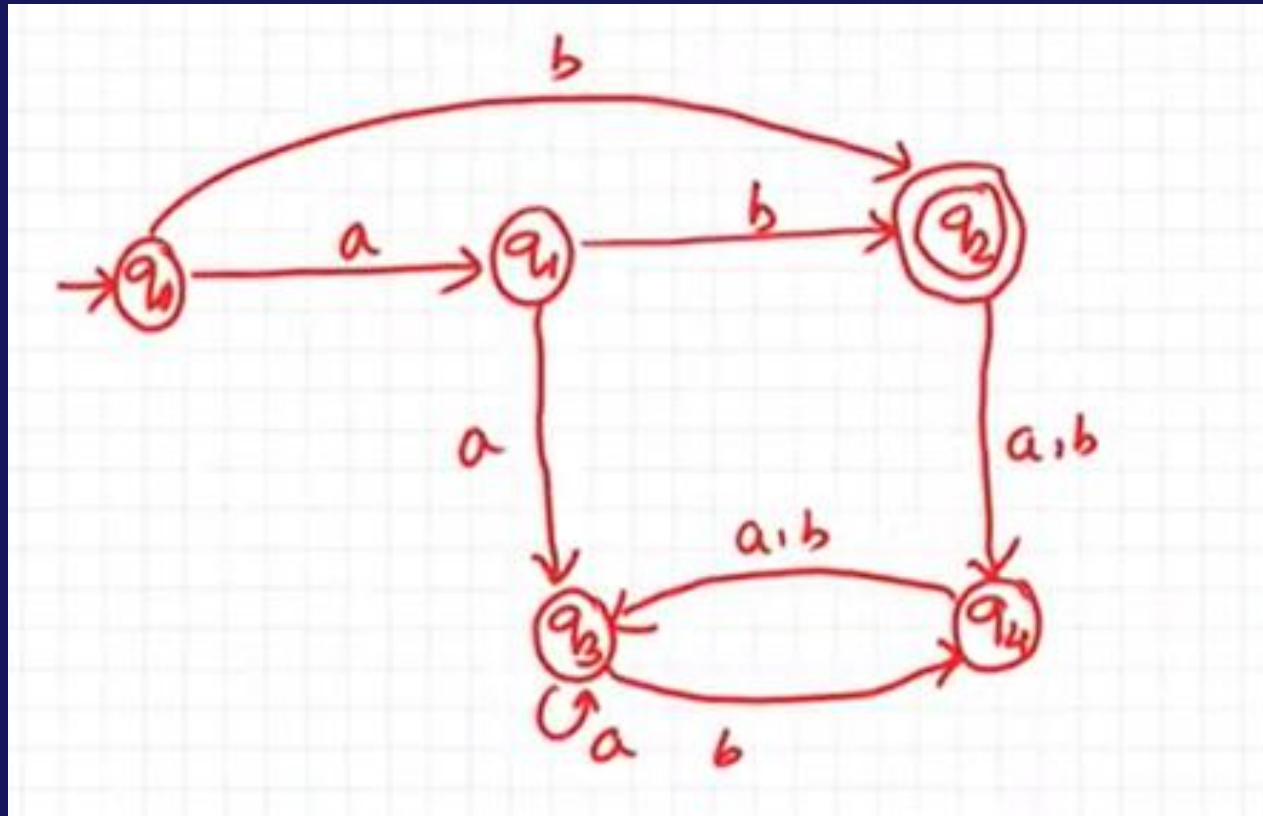
② Finiteness problem

Checking whether the given F.A. accepts finite or infinite language.

Algorithm

- S1. Eliminate all inaccessible states.
- S2. Delete the states from which we can not reach to the final state.
- S3. In the remaining F.A. if there is any cycle or loop then the F.A. accepts infinite language otherwise finite language.





3. Equivalence of two FA's:

Two finite automata over alphabet are equivalent if they accept the same set of strings over alphabet. when the two finite automata are not equivalent , there is some thing w over alphabet satisfying the following: one automaton reaches a FS on application of w , where as the other automaton reaches a non FS.

Comparison Method:

Let M and M' be two finite automata over alphabet. We construct a comparison table consisting of $n+1$ columns, where n is the number of i/p symbols.

The first column consists of pairs of vertices of the form (q, q') . Where

q belongs to M

q' belongs to M'

If (q, q') appears in some row of the first column, then the corresponding entry in the a -column (a belongs to alphabet) is (qa, qa') .

- The comparison table is constructed by starting with the pair of initial states of M and M' . We repeat the construction by considering the pairs in the second and subsequent column which are not in the first column.
- The row wise construction is repeated . There are two cases.

Case 1:

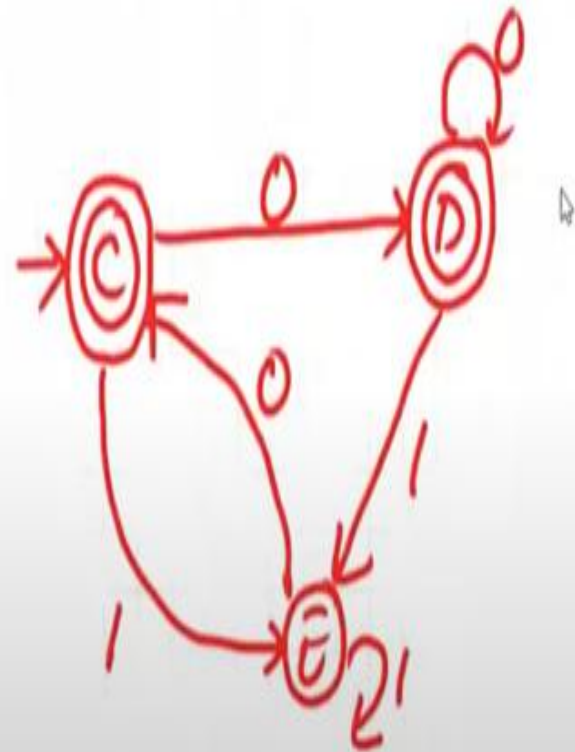
If we reach a pair (q, q') such that q is a FS of M and q' is a non FS of M' or viceversa , we terminate the construction and conclude that M and $M1$ are not equivalent.

Case 2:

The construction is terminated when no new element appears in the second and subsequent columns which are not in the first column. We conclude that M and M' are equivalent.

③ Equivalence problem

checking whether the given two F.A. accepts same language or not.



Closure Properties of Regular Sets:

A Closure Property is an Operation on languages that belong to a Class produces a result that is also in that same class.

REGULAR LANGUAGES ARE CLOSED UNDER REGULAR OPERATIONS

- **Closed Under Union:** $A \cup B = \{ w \mid w \in A \text{ or } w \in B \}$
- **Closed Under Concatenation:** $A \cdot B = \{ vw \mid v \in A \text{ and } w \in B \}$
- **Closed Under Kleene Star:** $A^* = \{ w_1 \dots w_k \mid k \geq 0 \text{ and each } w_i \in A \}$
- **Closed Under Intersection:** $A \cap B = \{ w \mid w \in A \text{ and } w \in B \}$
- **Closed Under complement:** $\neg A = \{ w \in \Sigma^* \mid w \notin A \}$
- **Closed Under Reversal:** $A^R = \{ w_1 \dots w_k \mid w_k \dots w_1 \in A \}$
- **Closed Under Homomorphism**
- **Closed Under Difference**

CLOSURE PROPERTIES

UNION THEOREM

Given two languages, L_1 and L_2 , define the **union of L_1 and L_2** as

$$L_1 \cup L_2 = \{ w \mid w \in L_1 \text{ or } w \in L_2 \}$$

Theorem: The union of two regular languages is also a regular language

Theorem : The regular sets are closed under union, concatenation & Kleene Closure

Proof:

If L_1 and L_2 are regular languages. There are regular expressions r_1 & r_2 that defines these languages. Then $(r_1 + r_2)$ is a regular expression that defines the language $L_1 + L_2$.

Example

- $L_1 = 0(0+1)^*$
- $L_2 = (0+1)^*1$
- $L_1 + L_2 = 0(0+1)^* + (0+1)^*1$

Proof contd....

- The language $L_1 \cdot L_2$ can be defined by $r_1 r_2$.
 - $L_1 = (0+1)$
 - $L_2 = 1(0+1)^*$
 - $L_1 L_2 = (0+1)1(0+1)^*$
 - The language L_1^* is defined by the regular expression r_1^* .
 - $L_1^* = (0+1)^*$
- Hence Proved**

Idea: If we can construct automata /regular expression for a language then it considered as regular language.

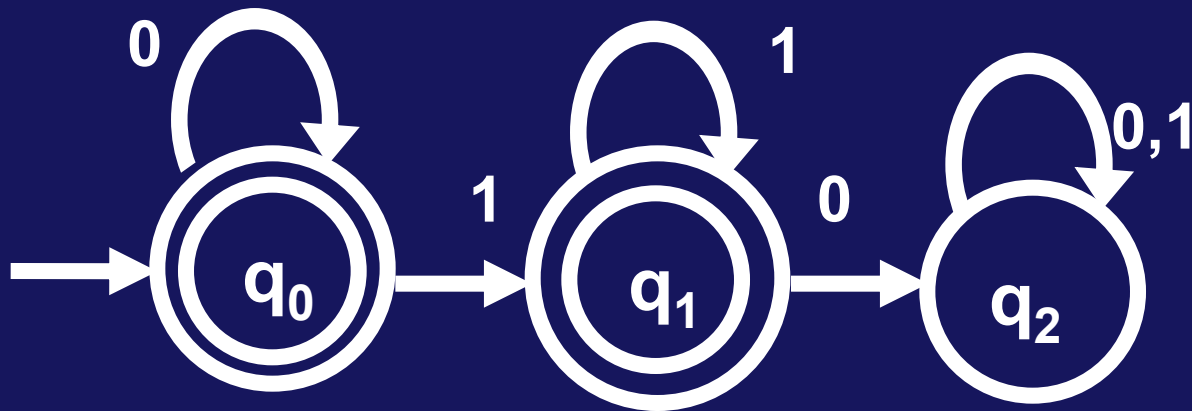
Theorem: The class of regular sets is closed under complementation.

Proof: Let

$M_1 = (Q, \Sigma, \delta, q_0, F)$ be finite automaton for L

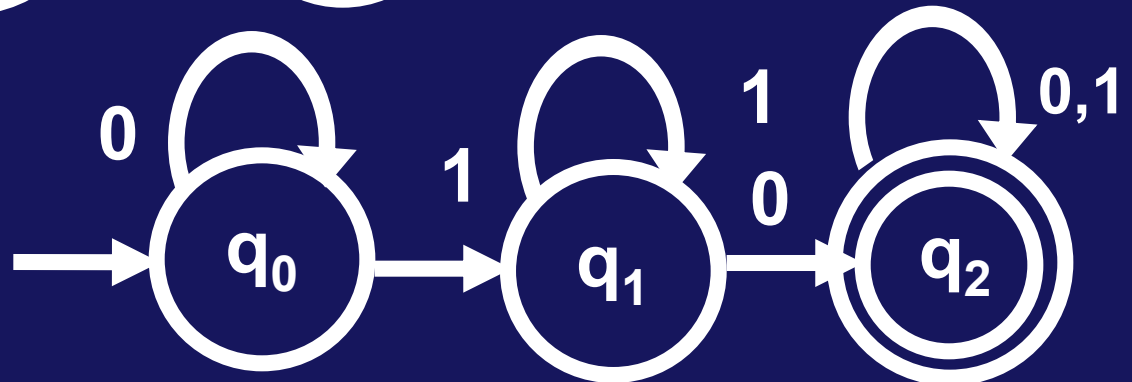
and

$M_2 = (Q, \Sigma, \delta, q_0, Q-F)$ be finite automaton for $\neg L$

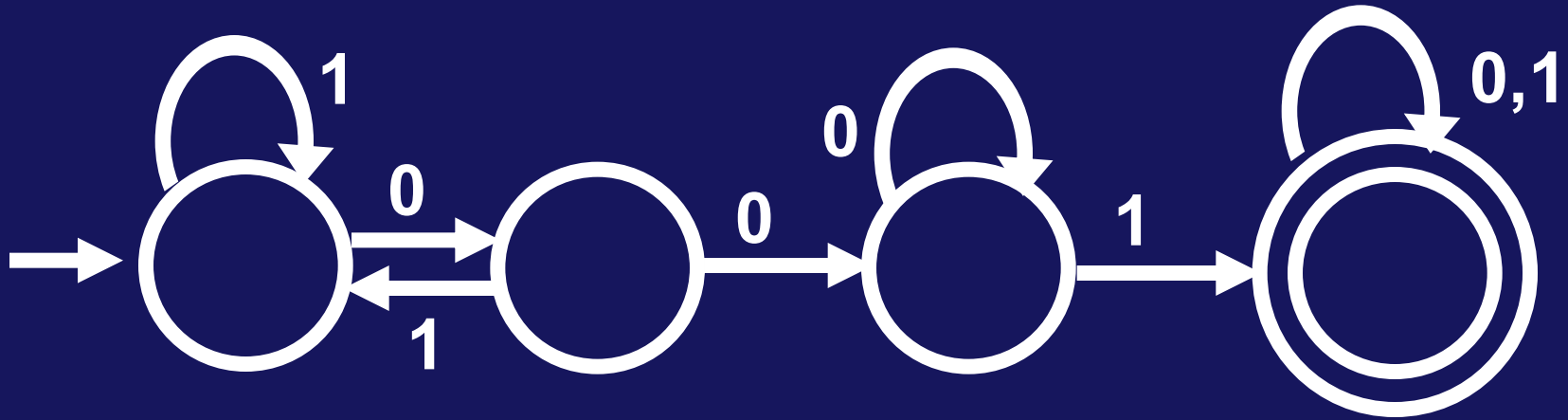


Does not contain 10

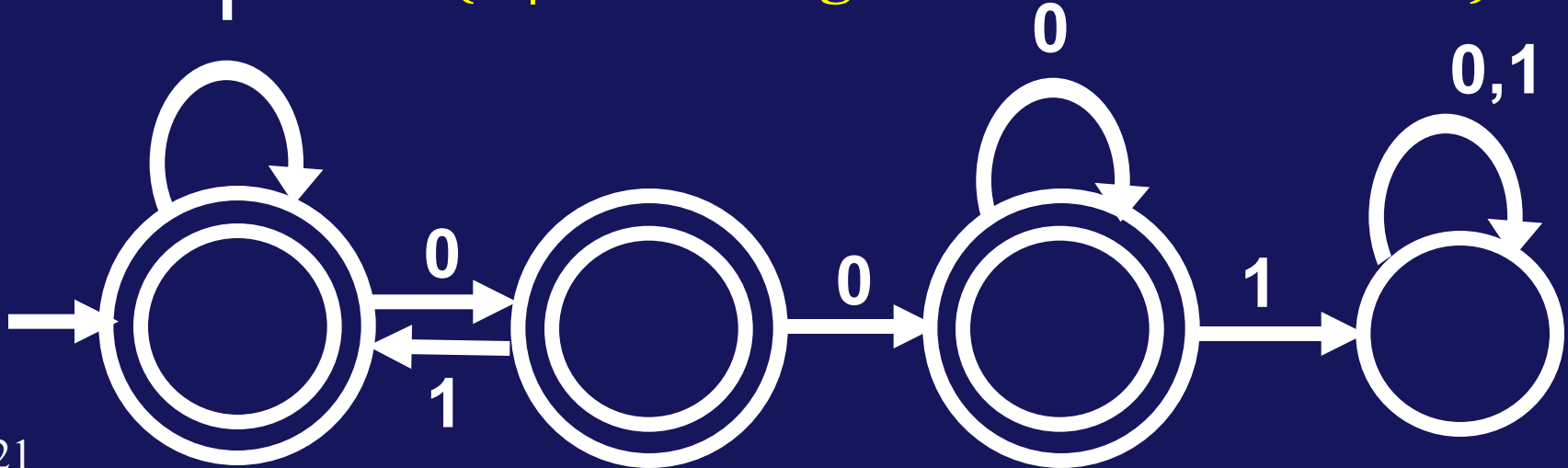
Does contain 10



$L = \{w \mid w = \text{string that contains } 001\}$



$\neg L = \{w \mid w = \text{string does not contain } 001\}$



Theorem: The regular sets are closed under intersection.

Proof: Let

$M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be finite automaton for L_1
and

$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be finite automaton for L_2

The intersection of two regular sets can be constructed by taking Cartesian product of states

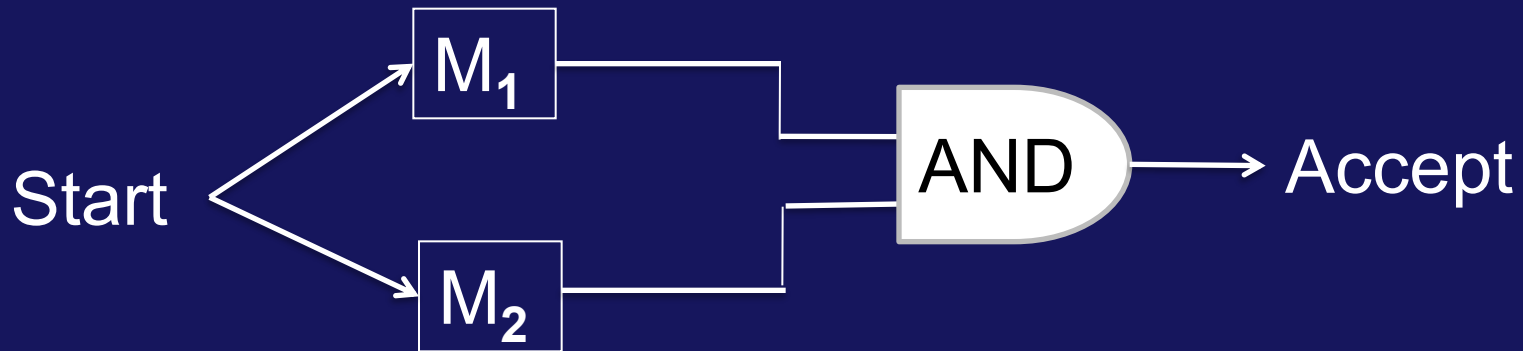
$$\begin{aligned} Q &= \text{pairs of states, one from } M_1 \text{ and one from } M_2 \\ &= \{ (q_1, q_2) \mid q_1 \in Q_1 \text{ and } q_2 \in Q_2 \} \\ &= Q_1 \times Q_2 \end{aligned}$$

Theorem: The regular sets are closed under Intersection contd..

$$\delta((q_1, q_2), \sigma) = (\delta_1(q_1, \sigma), \delta_2(q_2, \sigma))$$

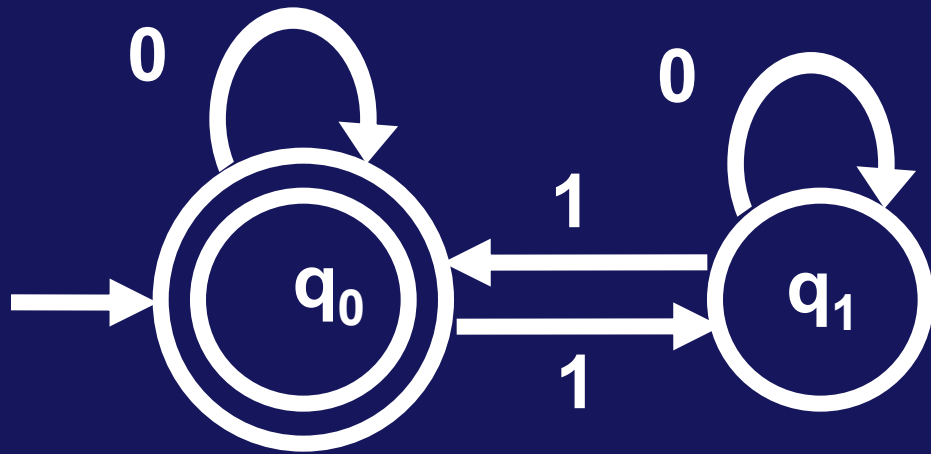
$$q_0 = (q_1, q_2)$$

$$F = \{ (q_1, q_2) \mid q_1 \in F_1 \text{ and } q_2 \in F_2 \}$$



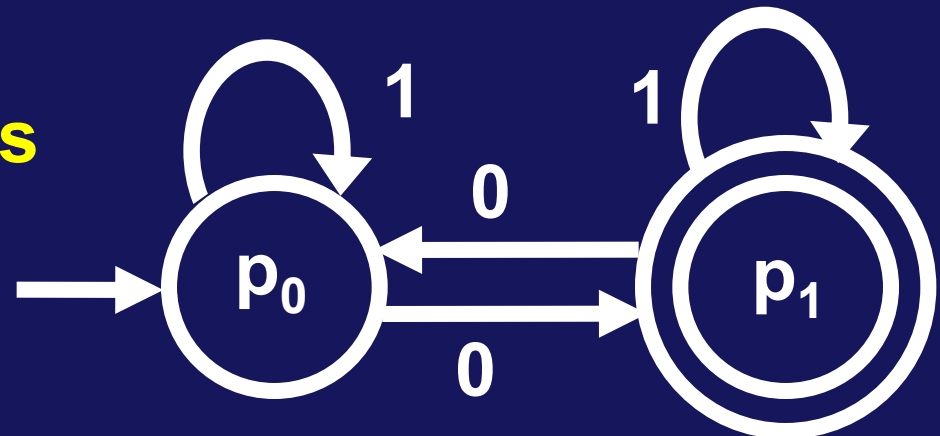
M accepts the Intersection of M_1 and M_2

Example: Even no. Of 1s intersection Odd number of 0s

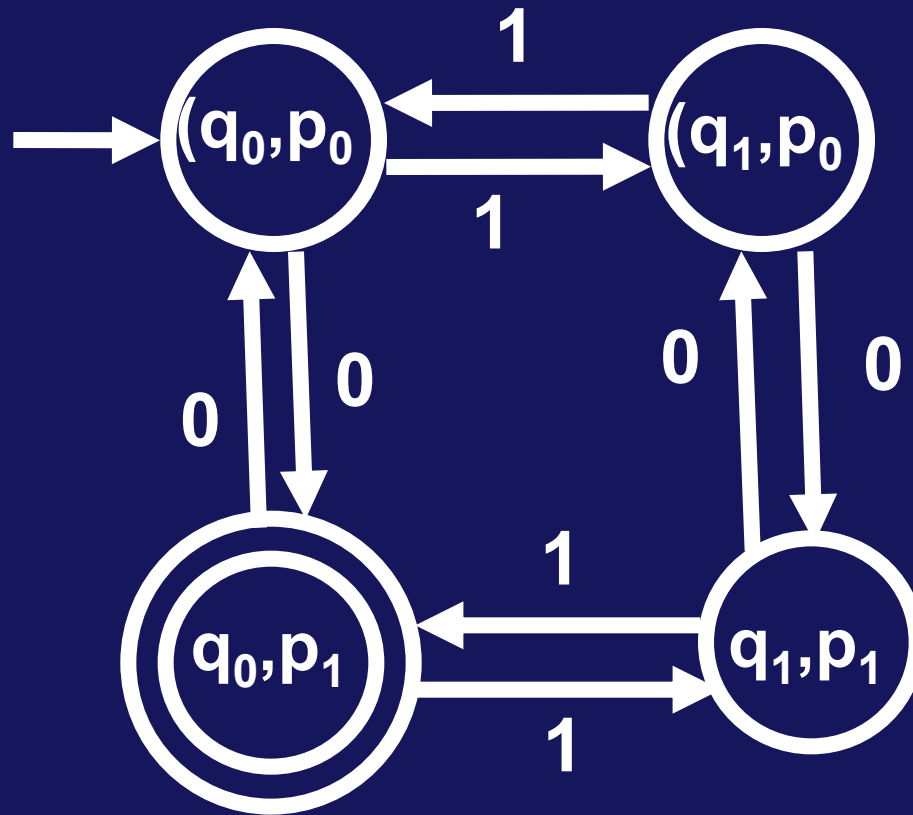


DFA for Even no. Of 1s

DFA for Odd number of 0s



DFA for Even no. of 1s intersection Odd number of 0s



Set Difference **THEOREM**

Given two languages, L_1 and L_2 , Set difference of L_1 and L_2 is defined as

$$L_1 - L_2 = L_1 \cap \neg L_2 = \{ w \mid w \in L_1 \text{ and } w \in \neg L_2 \}$$

Theorem: The set difference of two regular languages is also a regular language

Proof:

- It is proved that $\neg L_2$ is regular
- Also it proved $L_1 \cap \neg L_2$ is regular
- So $L_1 - L_2$ is regular.

REVERSAL THEOREM

If L is a regular language, then so is L^R .

Proof:

Assume L is defined by regular expression E . The proof is structured induction on the size of E .

Example: $w = a_1 a_2 \dots a_n$ $w^R = a_n a_{n-1} \dots a_1$.

BASIS: If E is ϵ , Φ or a for some symbol a then E^R is the same as E . $\epsilon^R = \epsilon$ $\Phi^R = \Phi$ $a^R = a$

Induction: There are three cases:

1. $E = E_1 + E_2$ $E^R = E_1^R + E_2^R$
2. $E = E_1 E_2$ $E^R = E_2^R E_1^R$
3. $E = E_1^*$ $E^R = (E_1^R)^*$

Example: $((0+1)0^*)^R = (0^*)^R (0+1)^R$
 $= (0^R)^* (0^R + 1^R)$
 $= 0^*(0+1)$

Homomorphism

- A string homomorphism is function on strings that works by substituting a particular string for each symbol.
- **Ex 1:** 0011 $h(0)=ab$ $h(1)=a$
- $0011=ababaa$
- **Ex 2:** Suppose h is the homomorphism from the alphabet $\{0,1,2\}$ to the alphabet $\{a,b\}$ defined by $h(0)=a$ $h(1)=ab$ $h(2)=ba$
- What is $h(0120)=?$
- $h(21120)=?$
- $h(01^*2)=?$

Homomorphism Theorem

- If L is a regular language over alphabet Σ and h is a homomorphism on Σ then $h(L)$ is also regular.
- Proof: Let $L=L(R)$ for some regular expression R . In general, if E is a regular expression with symbols in Σ .
- Let $h(E)$ be the expression we obtain by replacing each symbol a of Σ in E by $h(a)$. We claim that $h(R)$ defines the language $h(L)$.
- The proof is an easy structural induction.

Proof by structural Induction

- If E is expression of R and $h(E)$ is language after applying homomorphism
- $L(h(E))=h(L(E))$
- Basis: If $E = \epsilon$ or Φ then $h(E)=E$
- $L(h(E))=L(E)$; $h(L(E))=L(E)$
- $L(h(E))=L(E)=h(L(E))$
- if $E = a$ then $h(E)=h(a)$
- $L(h(E)) = \{h(a)\}$; $h(L(E)) = \{h(a)\}$
- $L(h(E)) = \{h(a)\} = h(L(E))$

Inductive Step

- There are three cases:

1. $E=F+G$ 2. $E=FG$ 3. $E=F^*$

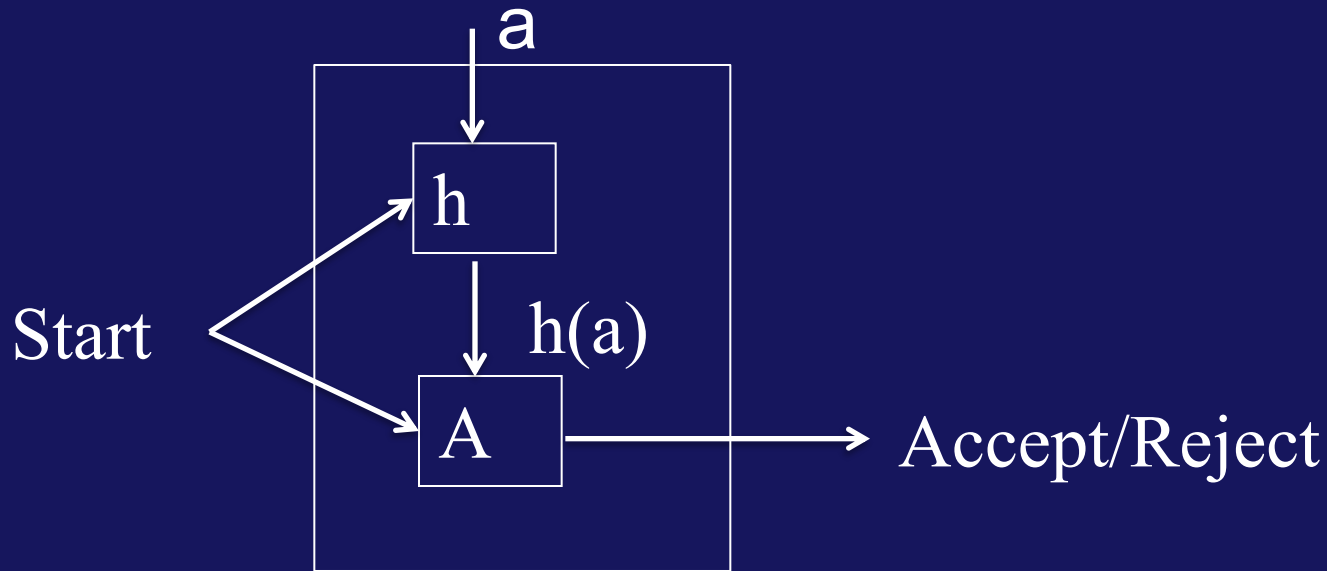
- If $E=F+G$ then $h(E)=h(F+G)=h(F)+h(G)$
- $L(E)=L(F) \cup L(G)$
- $L(h(E))=L(h(F)+h(G))=L(h(F)) \cup L(h(G))$
- $h(L(E))=h(L(F) \cup L(G))=h(L(F)) \cup h(L(G))$
- $L(h(F))=h(L(F))$
- $L(h(G))=h(L(G))$
- $L(h(E))=h(L(E))$

Hence Proved

Inverse Homomorphism

- Suppose h is a homomorphism from some alphabet Σ to strings in another alphabet T . Let L be language over alphabet T . Then $h^{-1}(L)$ is the set of strings w in Σ^* such that $h(w)$ is in L .
- **Theorem:** If h is a homomorphism from alphabet Σ to alphabet T and L is a regular language over T , then $h^{-1}(L)$ is also a regular language.
- We construct a DFA for $h^{-1}(L)$ from A and h .

DFA for $h^{-1}(L)$



Example: $L=ababa$ $h(0)=a$ $h(1)=ab$ $h(2)=ba$
 $h^{-1}(ababa)=022+110$
 $h^{-1}(a(ba)^*)=02^*$

This DFA uses the states of A but translates the input symbol according to h before deciding on the next state.

A language L is **regular** if it is **recognized** by a deterministic finite automaton (DFA), i.e. if there is a DFA M such that $L = L(M)$.

$L = \{ w \mid w \text{ contains } 001 \}$ is regular

$L = \{ w \mid w \text{ has an even number of } 1\text{'s} \}$ is regular

LANGUAGES - NOT REGULAR

$L = \{0^n 1^n \mid n \geq 0\}$ is NOT regular

$L = \{a^p \mid p \text{ is prime}\}$ is NOT regular

$L = \{a^{i^2} \mid i \geq 1\}$ is NOT regular

$L = \{0^{2^n} \mid n \geq 1\}$ is NOT regular

$L = \{0^n 1^m \mid n \geq m\}$ is NOT regular

$L = \{ww^R \mid w \in (a,b)^*\}$ is NOT regular

$L = \{ww \mid w \in (a,b)^*\}$ is NOT regular

$L = \{0^n 10^n \mid n \geq 0\}$ is NOT regular

$L = \{a^i b^j c^k \mid k < i+j, i, j \geq 1\}$ is NOT regular

THE PUMPING LEMMA (PL)

Let L be a regular language with $|L| = \infty$

Then **there exists a positive integer n** such that

if $w \in L$ and $|w| \geq n$

then $w = xyz$, where:

1. $|y| > 0$ or $y \neq \epsilon$
2. $|xy| \leq n$
3. $xy^kz \in L$ for any $k \geq 0$

Application of PL: It is used to prove that the language is not regular

Let M be a DFA that recognizes L

Let n be the **number of states in M**

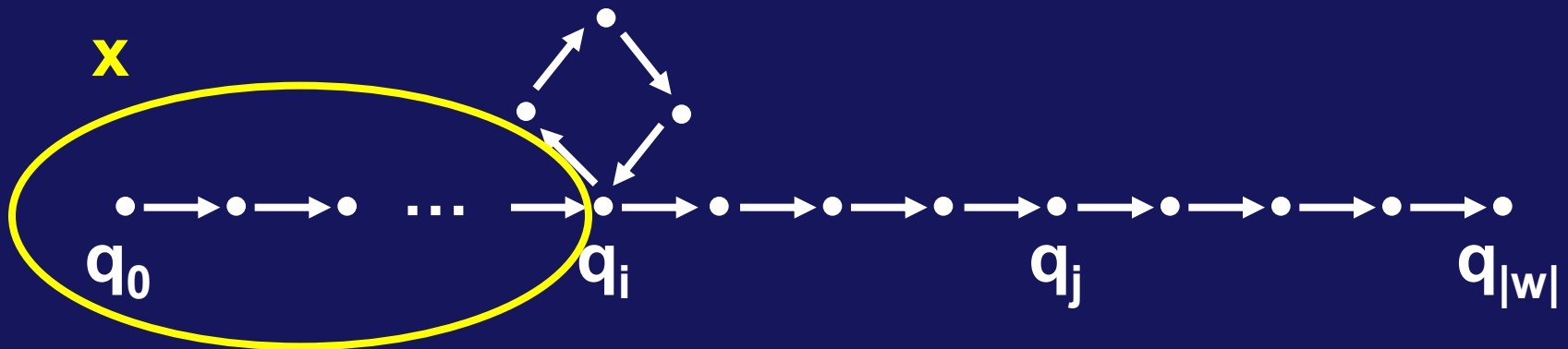
Assume $w \in L$ is such that $|w| \geq n$

We show $w = xyz$

1. $|y| > 0$

2. $|xy| \leq n$

3. $xy^kz \in L$ for any $k \geq 0$



There must be $j > i$ such that $q_i = q_j$

Prove not regular using pumping Lemma

$L = \{0^{2i} \mid i \geq 1\}$ is not regular

1. Assume L is regular. Let n be the number of states in finite automata accepting L .

2. Let $w = 0^{2i}$ Then $|w|=2i > n$.

3. By pumping lemma we can write $w=xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

Let $y = 0^q$. Then $|xz|= 0^{2i-q}$ (Length based proof)

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L .

$$\begin{aligned}
 |xy^kz| &= |xz| + k|y| \\
 &= (2i-q) + k(q) \\
 &= 2i + (k-1)q
 \end{aligned}$$

5. From pumping lemma it is clear $q \geq 1$

Therefore If $q=1$

$$|xy^kz| = 2i + (k-1)$$

For $k=0$ $|xy^0z| = 2i - 1 \notin L$.

Which is not the multiple of 2.

Therefore $L = \{0^{2i} \mid i \geq 1\}$ is not regular

Hence proved.

Prove $L = \{0^i 1^i \mid i \geq 1\}$ is not regular (PL)

Proof: (Structure based)

1. Assume L is regular. Let n be the number of states in finite automata accepting L .

2. Let $w = 0^i 1^i$ Then $|w|=2i > n$.

3. By pumping lemma we can write $w=xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

The string y can be in any of the following form:

a. y has only 0's $y = 0^q$ for some $q \geq 1$

b. y has only 1's $y = 1^r$ for some $r \geq 1$

c. y has both 0's and 1's $y = 0^q 1^r$ for some $q, r \geq 1$

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L.

Case a:

5. If $k=0$

$$xyz = 0^i 1^i \quad y = 0^q \quad xz = 0^{i-q} 1^i \notin L$$

Case b:

5. If $k=0$

$$xyz = 0^i 1^i \quad y = 1^q \quad xz = 0^i 1^{i-q} \notin L$$

Case c:

5. If $k=0$

$$xyz = 0^i 1^i \quad y = 0^q 1^r \quad xz = 0^{i-q} 1^{i-r} \notin L$$

Therefore $L = \{0^i 1^i \mid i \geq 1\}$ is not regular

Hence proved.

Prove not regular using pumping Lemma

$L = \{a^P \mid P \text{ is Prime}\}$ is not regular.

i.e. $L = \{a^2, a^3, a^5, a^7, a^{11}, \dots\}$

(or)

L consists of all strings of a 's whose length is a Prime

Proof: (Length based)

1. Assume L is regular. Let n be the number of states in finite automata accepting L .

2. Let $w = a^P$ Then $|w| = P \geq n+2$.

3. By pumping lemma we can break $w = xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

4. Let $|y| = m$. Then $|xz| = P - m$

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L.

5. If $k=P-m$

$$\begin{aligned} |xy^{P-m}z| &= |xz| + P-m|y| \\ &= (P-m) + P-m(m) \\ &= (P-m)(m+1) \text{ is not a prime.} \end{aligned}$$

Since it has two factors- $(P-m)$ and $(m+1)$.

We must check that neither of these factors are 1.

$$m+1 > 1 \text{ since } |y| \neq \varepsilon, m > 0 \quad (m+1) > 1$$

$$P-m > 1 \text{ since } P \geq n+2 \quad 1 \leq m \leq n$$

$$m = |y| \leq |xy| \leq n. \quad \text{So } (P-m) \geq 2.$$

(minimum value of P is n+2. maximum value of m is n)

since $xy^{P-m}z$ is not in the language, a^P is not regular

Hence proved.

Prove **$L = \{ a^{i^2} \mid i \geq 1 \}$ is not regular**

using pumping Lemma

i.e. $L = \{ a^1, a^4, a^9, a^{16}, a^{25}, \dots \}$

(or) L consists of all strings of a 's whose length is perfect square

Proof: (Length based)

1. Assume L is regular. Let n be the number of states in finite automata accepting L .

2. Let $w = a^{i^2}$. Then $|w| = i^2 \geq n$

3. By pumping lemma we can break $w = xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

4. Let $|y| = m$. Then $|xz| = i^2 - m$

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L .

$$\begin{aligned} |xy^kz| &= |xz| + k|y| \\ &= i^2 - m + k(m) \\ &= i^2 + (k-1)(m) \end{aligned}$$

5. If $k=0, m=1$ $|xy^0z| = i^2 - 1 \notin L$

This result leads to a contradiction to our assumption

since xy^0z is not in the language, a^{i^2} is not regular

Hence proved.

Prove $L = \{0^n 1^m \mid n \leq m \text{ and } n, m \geq 1\}$ is not regular using pumping Lemma

i.e. $L = \{0^1 1^1, 0^1 1^2, 0^1 1^3, \dots\}$

Proof: (structure based)

1. Assume L is regular. Let p be the number of states in finite automata accepting L .

2. Let $w = 0^n 1^m$ Then $|w| = n + m \geq p$

3. By pumping lemma we can break $w = xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

4. Let $y = 1^q$. Then $xz = 0^n 1^{m-q}$

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L .

$$\begin{aligned} xy^kz &= xz + k|y| \\ &= 0^n 1^{m-q} + k 1^q \end{aligned}$$

5. If $k=0$, $m=n$ $q=2$ $|xy^0z| = 0^n 1^{n-2} \notin L$

This result leads to a contradiction to our assumption

since xy^0z is not in the language, $0^n 1^m$ is not regular

Hence proved.

Prove $L = \{ww^R \mid w \in (a,b)^*\}$ is not regular using pumping Lemma

i.e. $L = \{aa, bb, abba, aaaa, bbbb, baab, \dots\}$

Proof: (Structure based)

1. Assume L is regular. Let p be the number of states in finite automata accepting L .

2. Let $w = a^n b^n b^n a^n$ Then $|w| = 4n \geq p$

3. By pumping lemma we can break $w = xyz$ such that

$$|y| > 0$$

$$|xy| \leq n$$

4. Let $xy = a^n$. $y = a^q$ Then $z = b^n b^n a^n$

4. Now consider the string xy^kz

If the given language is regular for all $k \geq 0$, the string xy^kz will be in L .

$$\begin{aligned} xy^kz &= xz + k|y| \\ &= a^{n-q} b^n b^n a^n + k \cdot 1^q \end{aligned}$$

5. If $k=0$, $q=2$ $|xy^0z| = a^{n-2} b^n b^n a^n \notin L$

This result leads to a contradiction to our assumption

since xy^0z is not in the language, ww^R is not regular

Hence proved.

Home work

1. Show that $L = \{a^n b a^n \mid n > 0\}$ is not regular
2. Show that $L = \{a^i b^j c^k \mid k > i + j \text{ and } i, j > 1\}$ is not regular